

Article

GA-Optimized Feature Weighting for Fuzzy C-Means Classification

Zhiwen Li ¹, Xinghua Wang ¹, Yongtao Li ¹ and Yubin Zhong ^{1,2,*}

¹ School of Mathematics and Information Science, Guangzhou University, Guangzhou 510006, China; lzhu2015@gzhu.edu.cn (Z.L.); 2112215082@e.gzhu.edu.cn (X.W.); 2112315061@e.gzhu.edu.cn (Y.L.)

² School of General Education, Guangzhou Institute of Science and Technology, Guangzhou 510540, China

* Correspondence: zhong_yb@gzhu.edu.cn

Abstract

The problem of insufficient classification accuracy in fuzzy clustering algorithms for multidimensional data is addressed in this paper. To tackle this issue, an improved genetic algorithm (GA)-based fuzzy controller is proposed, which combines the advantages of an improved genetic algorithm and a fuzzy C-means (FCM) clustering algorithm. The population initialization is performed using the Tent chaotic map, while the best individual retention strategy and last elimination selection operator are employed to adjust the population structure. Furthermore, an elitist crossover operator, an adaptive trial mutation operator, and a nonlinear convergence factor are introduced to mitigate the risk of falling into local optima. The controller algorithm integrates the intermediate parameters of FCM clustering into the fitness function of the genetic algorithm, and effectively improves the classification accuracy by screening the optimal feature subset and then fuzzy clustering. The experimental results on the Pistachio, WDBC, and Wine datasets show that the proposed method achieves competitive classification accuracy compared with other FCM-based feature-weighting optimization methods.

Keywords: genetic algorithm; fuzzy C-means clustering; fuzzy controller; tent chaotic map; classification algorithm

MSC: 68W50; 62H30; 90C59

1. Introduction

In the era of digital transformation, enterprises and organizations across diverse sectors—including smart cities, e-commerce, healthcare, and industrial manufacturing—are increasingly reliant on large-scale data collection to enhance user experiences, optimize operational workflows, and drive data-informed decision making [1,2]. The proliferation of high-dimensional datasets, characterized by numerous samples and extensive feature spaces, necessitates robust preprocessing techniques for data purification, feature screening, and intelligent classification to unlock their commercial and analytical value [3,4]. Clustering analysis, as a fundamental unsupervised learning technique, plays a pivotal role in discovering inherent data structures and grouping similar instances without prior label information [5].

Traditional clustering algorithms, such as k-means and hierarchical clustering, operate on the principle of “hard” classification, wherein each sample is exclusively assigned to a single cluster [6]. While computationally efficient, this crisp partitioning approach fails to adequately capture the inherent ambiguity and overlapping boundaries frequently observed in real-world datasets. In contrast, fuzzy C-means (FCM) clustering, originally



Academic Editor: Renato Bruni

Received: 3 June 2026

Revised: 7 July 2026

Accepted: 10 July 2026

Published: 18 July 2026

Copyright: © 2026 by the authors.

Licensee MDPI, Basel, Switzerland.

This article is an open access article distributed under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

proposed by Bezdek [7], offers a more flexible framework by assigning each sample a membership vector indicating its degree of belonging to multiple clusters simultaneously. This soft classification paradigm aligns more naturally with practical scenarios where data instances exhibit transitional or mixed characteristics. However, the conventional FCM algorithm suffers from several well-documented limitations: (i) sensitivity to initial cluster center selection, which frequently leads to convergence toward suboptimal local solutions [8]; (ii) computational inefficiency arising from the involvement of all sample points in each cluster center update iteration [9]; (iii) diminished clustering accuracy and robustness when handling high-dimensional or noisy data [10].

To mitigate these deficiencies, researchers have increasingly turned to metaheuristic optimization algorithms to enhance FCM performance. Genetic algorithms (GAs), pioneered by Holland in the 1970s, represent one of the most widely adopted evolutionary computation paradigms. GAs emulate natural selection mechanisms—including selection, crossover, and mutation—to explore complex search spaces and have demonstrated considerable success in global optimization tasks [11]. Nevertheless, standard GAs exhibit inherent weaknesses: inadequate local search capability, susceptibility to premature convergence, and parameter configurations that often rely heavily on empirical user experience rather than systematic guidance [12,13]. These limitations become particularly pronounced when GAs are applied to high-dimensional feature weight optimization problems in clustering contexts.

In recent years, the integration of chaotic maps with evolutionary algorithms has emerged as a promising strategy for enhancing population diversity and convergence behavior. Chaotic systems, characterized by ergodicity, randomness, and sensitivity to initial conditions, have been successfully employed to initialize populations in various metaheuristic frameworks. For instance, the Logistic map, Tent map, and Sine map [14,15] have been incorporated into particle swarm optimization (PSO), differential evolution (DE), and genetic algorithms to improve solution quality and avoid premature convergence. Specifically, chaotic-map-enhanced GAs have demonstrated superior performance in function optimization [16], feature selection [17], and clustering applications [18] compared to their random-initialization counterparts. The Tent chaotic map, in particular, has attracted significant attention due to its uniform distribution properties and enhanced ergodicity relative to the Logistic map, making it well-suited for generating diverse initial populations in genetic algorithms [19].

The hybridization of metaheuristic algorithms with FCM clustering has attracted substantial attention in recent years. While early studies explored PSO-FCM [20], ABC-FCM [21], WOA-FCM [22], and SSA-FCM [23] hybrids, significant advances have emerged since 2022. Qi et al. [24] and Liu et al. [25] demonstrated that tent chaotic initialization achieves superior population diversity and uniform distribution over random initialization, with Rauf et al. [26] extending this to multi-population differential evolution frameworks. Wang et al. [27] further established that tent mapping exhibits better spatial distribution than Logistic and Sine maps, reducing premature convergence. In feature-weighted fuzzy clustering, Askari et al. [28] conducted a comprehensive survey (2004–2023) categorizing local and global weighting schemes, while Liu et al. [29] showed that composite fitness functions integrating intra-cluster compactness and inter-cluster separation outperform single-metric objectives.

Despite these advances, several gaps remain in the existing literature. First, while chaotic initialization has been applied to various metaheuristic frameworks, its systematic integration with matrix-structured population representations in genetic algorithms remains underexplored. Second, most existing FCM-metaheuristic hybrids employ fitness functions that focus solely on clustering accuracy, neglecting the geometric relationships

among cluster centers and intra-cluster compactness that fundamentally determine clustering quality [16]. Third, the adaptive control of mutation intensity through nonlinear convergence factors perturbed by probability distributions has received limited attention in the context of genetic algorithm-based feature weighting for FCM.

This study proposes a novel fuzzy controller based on an enhanced genetic algorithm (MGA + FCM) to address existing research gaps, with three key contributions. First, the Tent chaotic map is employed for population initialization to ensure superior diversity and uniform distribution of initial solutions, while a matrix-structured population representation is introduced alongside a best individual retention strategy and last elimination selection operator to dynamically optimize the population structure throughout the evolutionary process. Second, an elitist crossover operator is designed to preserve and propagate high-quality solutions, complemented by an adaptive tentative mutation operator that adjusts mutation intensity based on individual fitness relative to the population; a nonlinear convergence factor perturbed by beta-distributed random numbers is also introduced to dynamically balance global exploration in early iterations with local exploitation in later stages, effectively mitigating premature convergence and local optima entrapment. Third, a composite fitness function integrating FCM classification accuracy with inter-cluster distance maximization and intra-cluster radius minimization is proposed, enabling simultaneous feature subset screening and fuzzy clustering optimization through a feedback mechanism that enhances overall classification performance. Experimental validation on the Pistachio, WDBC, and Wine datasets demonstrates that the proposed MGA + FCM method achieves competitive classification performance compared with FA + FCM. MGA + FCM obtains higher average accuracy on the Pistachio and WDBC datasets and shows comparable performance on the Wine dataset, indicating that the proposed enhanced genetic algorithm can effectively search for feature-weight solutions under different data characteristics.

The remainder of this paper is organized as follows. Section 2 establishes the fuzzy clustering index system and analyzes the dynamic clustering characteristics of the dataset. Section 3 details the proposed enhanced genetic algorithm, including population initialization, structure representation, genetic operators, and the FCM-integrated fitness function. Section 4 presents experimental results and comparative analysis. Section 5 concludes with a discussion of contributions and future research directions.

2. Establishment of Fuzzy Clustering Index System

The fuzzy clustering analysis model is a prevalent mathematical classification model in fuzzy mathematics. It constructs a dynamic fuzzy clustering graph by sorting the fuzzy equivalent matrix $t(R)$ according to the confidence level $\lambda \in [0, 1]$. A higher value of λ means that the data features are more informative for classification and comparison [19].

Taking the pistachio binary classification dataset used in this paper as an example, we first establish a feature set $Q = \{k_1, k_2, k_3, \dots, k_{14}, k_{15}, k_{16}\}$ corresponding to the 16 data features, as shown in Table 1.

Secondly, to construct the fuzzy dynamic fuzzy clustering graph and analyze the relationships among the 16 features, we randomly selected 200 samples (100 from each class) from the dataset to ensure class balance, and constructed a 200×16 initial data matrix X . This balanced subset is used exclusively for establishing the fuzzy clustering index system and analyzing feature relationships. Representative data are presented in Table 2.

The initial matrix X is normalized to yield matrix T . The corresponding formulation is presented below:

$$t_{ij} = \frac{x_{ij} - \min(x_i)}{\max(x_i) - \min(x_i)} \quad (1)$$

Table 1. Data feature correspondence table.

Symbol	Description
k_1	Area
k_2	Circumference
k_3	Major_axis
k_4	Minor_axis
k_5	Distance of eccentricity
k_6	Equivalent diameter
k_7	Solidity
k_8	Area of convex hull
k_9	Extent
k_{10}	Ratio of axial length
k_{11}	Degree of approximation to a circle
k_{12}	Compactness
k_{13}	Form factor 1
k_{14}	Form factor 2
k_{15}	Form factor 3
k_{16}	Form factor 4

Subsequently, following the fuzzy clustering theory, the normalized matrix T is fuzzified to generate the fuzzy similarity matrix $R = (r_{ij})$, where r_{ij} denotes the similarity coefficient between features k_i and k_j .

Table 2. Partial initial matrix feature data table.

k_1	k_2	k_3	...	k_{14}	k_{15}	k_{16}
60,955	999.789	386.9247	...	0.0034	0.5184	0.9591
96,395	1352.674	515.873	...	0.0026	0.4612	0.9648
91,533	1915.9561	451.7373	...	0.0029	0.5711	0.9629
$\vdots$	$\vdots$	$\vdots$	...	$\vdots$	$\vdots$	$\vdots$
106,889	1267.8781	506.6975	...	0.0025	0.5301	0.995
82,130	1169.217	466.5165	...	0.0028	0.4805	0.9719
84,557	1138.902	416.459	...	0.0031	0.6207	0.9868

The max-minimum method is adopted here as the standard fuzzification technique in fuzzy mathematics. The corresponding formula is presented below:

$$r_{ij} = \frac{\sum_{k=1}^m (t_{ik} \wedge t_{jk})}{\sum_{k=1}^m (t_{ik} \vee t_{jk})} \quad (2)$$

where $\wedge$ and $\vee$ denote the minimum and maximum operators, respectively (i.e., $\wedge = \min(x, y)$ and $\vee = \max(x, y)$), and m represents the number of rows in the normalized matrix T [30]. Using the formula above, the fuzzy similarity matrix R between features is constructed. R is a square matrix whose dimension equals the number of data features. Representative entries are presented in Table 3.

Fuzzy similarity matrix R is reflexive and symmetric, but not transitive. According to fuzzy clustering principle, R cannot be directly employed for clustering analysis.

Therefore, the fuzzy equivalence matrix $t(R)$ is constructed from R via the transitive closure method. This method involves iteratively squaring the similarity matrix and checking whether R^{2^n} equals R^n . If this equality holds, $t(R)$ is defined as R^n . Otherwise, the squaring process continues until convergence is reached [31,32]. In fuzzy matrix multiplication, the conventional multiplication and addition operations are replaced by

the minimum ($\wedge$) and maximum ($\vee$) operators, respectively. Representative entries of the resulting fuzzy equivalence matrix $t(R)$ are presented in Table 4.

Table 3. Partial data of fuzzy similarity matrix.

1.0000	0.4812	0.8352	...	0.3628	0.8333	0.5940
	1.0000	0.4574	...	0.5340	0.4816	0.3150
		1.0000	...	0.3328	0.7338	0.6450
$\vdots$	$\vdots$	$\vdots$	...	$\vdots$	$\vdots$	$\vdots$
			...	1.0000	0.3829	0.2299
	Symmetry		...		1.0000	0.5953
			...			1.0000

Finally, once the fuzzy equivalence matrix $t(R)$ is obtained, fuzzy clustering analysis is performed on the data features by decreasing the confidence level λ from large to small, yielding the dynamic fuzzy clustering graph shown in Figure 1.

Table 4. Partial data of fuzzy equivalence matrix.

1.0000	0.5719	0.8608	...	0.5340	0.8587	0.7834
	1.0000	0.5919	...	0.5340	0.5719	0.5719
		1.0000	...	0.5340	0.8587	0.7834
$\vdots$	$\vdots$	$\vdots$	...	$\vdots$	$\vdots$	$\vdots$
			...	1.0000	0.5340	0.5340
	Symmetry		...		1.0000	0.7834
			...			1.0000

The dynamic fuzzy clustering graph characterizes the similarity and grouping relationships among data features across various confidence levels. Features clustered at higher confidence levels exhibit stronger similarity, whereas those merged at lower levels display relatively weaker affinities. As such, the graph serves as a qualitative reference for understanding feature interrelationships and identifying potential redundancy.

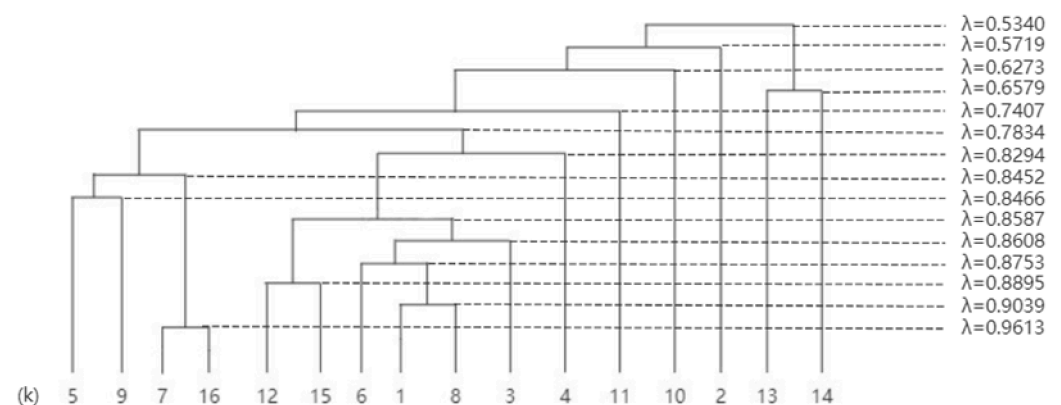


Figure 1. Dynamic fuzzy clustering graph.

3. Establishment of FCM Model Based on Enhanced Genetic Algorithm

3.1. Binary Encoding Is Used for Feature Weight Representation

There are three common coding methods in a genetic algorithm: binary encoding, decimal coding and real-valued encoding. Binary encoding is more convenient than the other two for population crossover and mutation operations. When the value range of the weights is small, the precision of binary encoding is comparable to that of decimal

and real-valued encoding [33]. The core idea of the proposed algorithm is to search for an optimal set of feature weights, denoted as W . Initially, the dataset is scaled to maximize the corresponding fitness function value. The scaling formulation is given in Equation (3):

$$\begin{aligned} w \otimes x &= (w_1, w_2, \dots, w_n) \otimes (x_1, x_2, \dots, x_n) \\ &= (w_1 x_1, w_2 x_2, \dots, w_n x_n) \end{aligned} \quad (3)$$

where $\otimes$ represents the scaling operator for population individual data, w denotes the feature weight vector, and x_i refers to the i -th feature column of the dataset. Each weight value w_i in the feature weight vector falls within the range $[0, 1]$ and is represented using a 20-bit binary encoding, yielding a precision of 10^{-6} .

3.2. The Tent Chaotic Map Initializes the Population

Chaotic maps are widely used in swarm intelligence algorithms. Using chaotic maps instead of random initialization enables the algorithm to generate initial solutions with superior diversity and uniform distribution across the problem search space. A relatively uniform initial solution prevents premature convergence to sub optimal regions, thereby facilitating the discovery of high-quality solutions. The contribution of a high-quality initial population to the convergence speed and solution accuracy of swarm intelligence algorithms is significant and should not be overlooked.

The Tent chaotic map initializes the genetic population as follows:

Step 1: Determine the value of the parameter.

Step 2: Set the value range of the initial population and the population number num according to the optimization problem, and randomly generate m values within this range to form the sequence X_0 . m is the dimension of the search space.

Step 3: Let $X_0 = x_0(n), n = 1, 2, \dots, m$.

Step 4: Let $x_0(1) = X_0$, $x_0(n+1)$ be calculated by Formula (4):

$$x(n+1) = \begin{cases} \frac{x(n)}{\alpha}, & x(n) \in [0, \alpha], n+1 \leq num \\ \frac{1 - \frac{x(n)}{\alpha}}{(1-\alpha)}, & x(n) \in [\alpha, 1], n+1 \leq num \end{cases} \quad (4)$$

where $0 < \alpha < 1$, $\alpha \neq 0.5$, $\alpha \neq x_0$.

Step 5: Save the sequence x and concatenate the corresponding positions of m sequences x to form num m -dimensional initial solutions.

In this paper, α is set to 0.75. The tent chaotic map is shown in Figure 2.

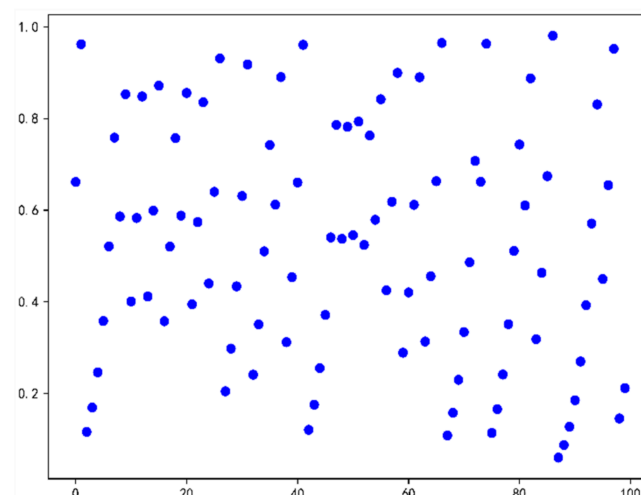


Figure 2. Uniform distribution diagram of tent map.

3.3. Population Structure Representation

The population structure of the enhanced genetic algorithm is a matrix structure. Each element in the matrix represents an individual feature-weight vector, denoted as W_{ij} , where i and j denote the row and column positions of the individual in the population matrix, respectively. For a dataset with L features, each W_{ij} corresponds to a $1 \times L$ feature-weight vector. Since each feature weight is represented by a 20-bit binary encoding, each individual W_{ij} is encoded as a $1 \times (20L)$ binary vector composed of $\{0, 1\}$. For example, for the Pistachio dataset with 16 features, each individual is encoded as a 1×320 binary vector.

In the matrix structure, the individuals in the same row are defined as a row population, the individuals in the same column are defined as a column population, and the individuals on the main diagonal are defined as the main diagonal population. The matrix-structured population enables the algorithm to perform row-based search, elite retention, and crossover operations according to the spatial position of individuals in the matrix. The specific structure is shown in Figure 3.

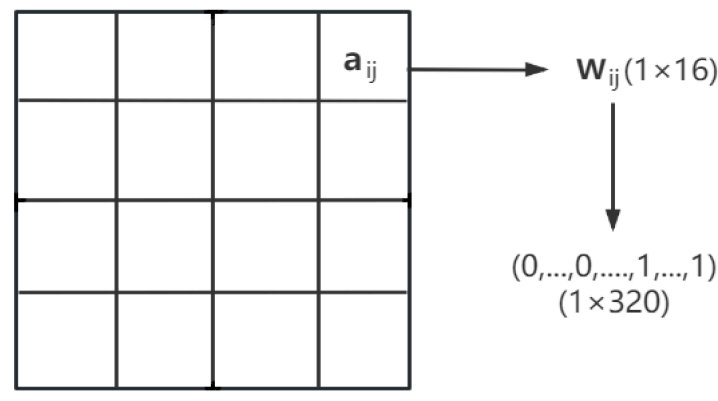


Figure 3. Population structure.

3.4. Row Search Operator and Move Operator

After population initialization, the fitness value of each population was calculated according to the fitness function. The individual with the highest fitness value in each row population was placed in the position of the corresponding row position in the main diagonal of the matrix, while the individual with the worst fitness value in the row was placed adjacent to the optimal population. The specific operation was implemented by the pseudo-code *rebuild(pop)*, which proceeds as follows: the worst individual in the first row of the matrix is placed to the right of the optimal individual; the worst individual in the last row is placed to the left of the optimal individual; and the two worst individuals from the remaining rows are placed on both sides of the optimal individual, respectively.

3.5. Elite Crossover Operator

Crossover is the primary mechanism for generating new individuals in genetic algorithms. The proposed crossover operator selects elite individuals from the main diagonal for pairwise recombination, where each elite individual is crossed only with its adjacent counterpart in the adjacent row. Specifically, for a population matrix of order N , there are N elite populations, yielding $(N - 1)$ crossover operations and producing $2(N - 1)$ offspring. These offspring are placed on both sides of the dominant individual to replace the previously screened worst individuals. The specific operation is implemented by pseudo-code *cross_DNA*($a_{i,i}, a_{i+1,i+1}, a_{i,i+1}, a_{i+1,i}, \text{cross_position}$), that is, the elite populations $a_{i,i}$ and $a_{i+1,i+1}$ are crossed and exchange the crossover point *cross_position*, generating new individuals that are placed in positions $a_{i,i+1}$ and $a_{i+1,i}$.

3.6. Adaptive Tentative Mutation

Population mutation is the secondary operator for generating new individuals, with a probability traditional much lower than that of crossover. Tentative mutation means that only when the fitness value of the population after mutation is greater than the fitness value of the population before mutation will a new population be generated to replace the population before mutation. A tentative mutation occurs in each population in the matrix, and the specific operation is realized by pseudo-code *mutate(pop, mutate_default_num, mostfit, lastfit)*. The number of mutation sites is adaptive [34], with sites randomly selected and bits flipped (0 to 1, or 1 to 0). The corresponding formula is given below:

$$num = \left\lfloor \frac{mfit - fit}{mfit - lfit + 0.0001} \times A \times defm \right\rfloor \quad (5)$$

where *num* denotes the number of mutation sites for the target individual, *mfit* represents the fitness value of the best individual in the current generation's population matrix, *lfit* represents the fitness value of the worst individual in the current generation's population matrix, and *defm* is a user-defined integer parameter initialized to 150.

$$A = 2 / (1 + \exp(20t / T_{max} - 10) + \sigma f(p, q)) \quad (6)$$

where $p, q \in (0, 1)$.

Traditionally, the linear variation of parameter *A* tends to significantly degrade the algorithm's search performance. To address this issue, the formula for calculating *A* is reformulated, and a probability density function $f(\cdot)$ is introduced to perturb *A*, thereby balancing its ability to escape from local optima. In the equation, σ represents the adjustment factor, which is set to 0.15 in this paper; $f(\cdot)$ denotes the probability density function of the beta distribution, used to generate a set of random numbers conforming to the beta distribution; the parameters p and q are independently generated from the uniform distribution $U(0, 1)$, i.e., $p, q \in (0, 1)$; T_{max} represents the maximum number of iterations of the improved genetic algorithm, which is set to 300 in this study. Figure 4 illustrates the comparison of parameter *A* before and after the proposed improvement.

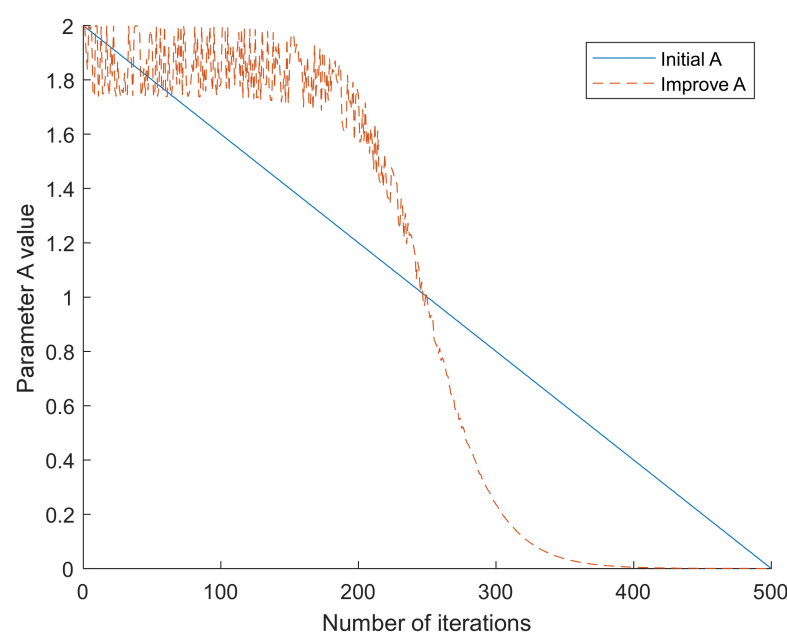


Figure 4. Parameter *A* before and after improvement.

According to Formula (5), individuals with lower fitness values undergo mutation at more sites, resulting in greater variations; conversely, individuals with higher fitness values experience smaller perturbations. The improved parameter A follows a nonlinear decreasing trend: it decreases slowly during the initial optimization stage, thereby enhancing global exploration; drops rapidly in the middle stage to ensure convergence speed; and gradually approaches zero in the later stages, improving local exploitation while preventing premature convergence to local optima. Furthermore, the beta-distributed random numbers generated by $f(p, q)$ are utilized to locally perturb A , dynamically adjusting the step size at each iteration and thereby increasing the diversity of solutions.

3.7. Population Fitness Function

The fitness function of the enhanced genetic algorithm incorporates not only the classification accuracy of fuzzy C-means (FCM) clustering, but also the relationship of the cluster centers and the radius of each cluster in the FCM algorithm. Each individual in the population represents a candidate feature-weight solution w_{ij} . Applying the scaling operator yields the corresponding weighted data matrix, on which the FCM algorithm is subsequently executed to obtain the clustering-based classification accuracy and the intermediate parameters for that individual. The fitness function is formulated as follows:

$$fit = acc + \frac{\sum_{i=1}^{m-1} \sum_{j=i+1}^m L(C_i, C_j)}{q \times \frac{m(m-1)}{2}} + e^{-\sum_{i=1}^m R_i} \quad (7)$$

where acc denotes the classification accuracy of the FCM algorithm performed on the scaled dataset $X_{ij} = \{w_{ij1}x_1, \dots, w_{ij16}x_{16}\}$, C_i represents the cluster center of the i th class, q denotes the number of features, and m denotes the number of cluster centers. The function $L(C_i, C_j)$ represents the Euclidean distance between cluster centers C_i and C_j , and R_i denotes the Euclidean distance between the farthest point in each cluster and the corresponding cluster center.

According to Equation (7), the fitness value is directly proportional to the FCM classification accuracy and the inter-cluster distance, and inversely proportional to the intra-cluster radius. Equation (7) constrains the three components to a combined range of $[0, 3]$.

It should be noted that the clustering-based classification accuracy used in the fitness function is calculated with reference to the known class labels. Therefore, the proposed MGA + FCM method is not a fully unsupervised clustering method, but a supervised feature-weighting scheme built upon FCM. The class labels are used to evaluate the fitness of candidate feature-weight solutions and guide the optimization process.

3.8. Algorithm Steps

Step 1: Tent chaotic map initializes the matrix structure population $P(t)$, where t is 0, $P(t)$ is composed of N row populations $P_0, P_1, \dots, P_{N-1}$, and the size of each row population is N . a_{ij} is used to represent the element in row i and column j of matrix structure population.

Step 2: For each row population P_i , the optimal population is found according to the fitness function to exchange with a_{ii} , and the worst population is found next to a_{ii} according to the rules.

Step 3: For the population in the main diagonal position $\{a_{ii} | i = 1, \dots, N-1\}$, two adjacent populations are crossed to generate a new population on both sides of the diagonal.

Step 4: For each population, copy the population a_{ij} to temporary individual $Temp$. Individual $Temp$ carries out adaptive tentative mutation. If the fitness value after mutation was better than that of population a_{ij} , $Temp$ is assigned to population a_{ij} .

Step 5: Check whether t meets the number of iterations; otherwise $t = t + 1$, and jump to Step 2.

Step 6: Under the nested cross-validation scheme, the optimal population is selected from the main diagonal population set $\{a_{ii} | i = 1, \dots, N - 1\}$ only within the training fold, and the binary weights are transformed into the corresponding decimal feature-weight vector $W = (w_1, w_2, \dots, w_L)$. According to the threshold method, the effective feature weights are selected, and the training data set is weighted as $X_{train}^W = (w_1x_1, w_2x_2, \dots, w_Lx_L)$ during the optimization process. After the optimal feature-weight vector W^* is obtained from the training fold, it is fixed and applied to the corresponding test fold to obtain $X_{test}^W = (w_1x_1, w_2x_2, \dots, w_Lx_L)$. Finally, the FCM algorithm is used on the weighted test data, and the accuracy of the test fold is calculated. The average accurate value over all folds is calculated as the final classification result of the fuzzy controller.

4. Model Analysis and Results

4.1. Experimental Dataset Characteristics

All experiments were conducted on a computer equipped with an Intel Core i5-7300 HQ processor, 8 GB of RAM, and an NVIDIA GeForce GTX 1050 Ti graphics processing unit. The operating system was Windows 10, and the proposed algorithm was implemented in Python 3.11. Unless otherwise specified, all algorithms were executed under identical hardware and software configurations to ensure a fair comparison.

To further evaluate the applicability of the proposed method, three datasets were used in the experiments, including the Pistachio dataset, the Wisconsin Diagnostic Breast Cancer (WDBC) dataset, and the Wine dataset [35–37]. The Pistachio dataset contains 1718 samples and 16 features, including 998 Kirmizi samples and 720 Siit samples. The WDBC dataset contains 569 samples and 30 features, and the Wine dataset contains 178 samples and 13 features. These datasets include both binary and multi-class classification tasks, which can provide a broader evaluation of the proposed method. The 200 samples mentioned in Section 2 were used only for constructing the fuzzy dynamic clustering graph of the Pistachio dataset, whereas the algorithm comparison and cross-validation experiments in this section were performed on the complete datasets.

To avoid potential data leakage during feature-weight optimization, a nested cross-validation scheme was adopted in the experimental evaluation. Specifically, a stratified 5-fold cross-validation procedure was used to preserve the class distribution of each dataset in each fold. In each fold, approximately 80% of the samples were used as the training set, and the remaining 20% were used as the test set. For example, for the Pistachio dataset with 1718 samples, each training fold contained approximately 1374 samples, while each test fold contained approximately 344 samples.

For each fold, the feature-weight optimization process was performed only on the training set. During this process, the candidate feature-weight solutions were evaluated by the fitness function using the training data, and the test set was not involved in population initialization, fitness calculation, genetic evolution, or feature-weight selection. After the optimal feature-weight vector was obtained from the training set, it was fixed and then applied to the corresponding test set. The FCM algorithm was subsequently performed on the weighted test data, and the classification accuracy was calculated on the test set.

To reduce the influence of randomness, the above 5-fold cross-validation procedure was repeated independently three times. The same validation protocol was applied to all compared algorithms to ensure a fair comparison.

4.2. Experimental Results and Analysis

The effectiveness of the proposed MGA + FCM method was evaluated using the nested cross-validation scheme described above. Each algorithm was independently run 3 times with the same population size, number of iterations, and validation protocol. The average classification accuracy is reported in Table 5.

Table 5. Classification accuracy comparison on three datasets.

Dataset	Iterations	Dimension	Algorithm	Accuracy (%)
Pistachio type detection	300	16	MGA + FCM	86.61 ± 0.06
			FA + FCM	86.44 ± 0.02
WDBC	300	30	MGA + FCM	97.31 ± 0.33
			FA + FCM	97.07 ± 0.08
Wine	300	13	MGA + FCM	98.13 ± 0.70
			FA + FCM	98.31 ± 0.46

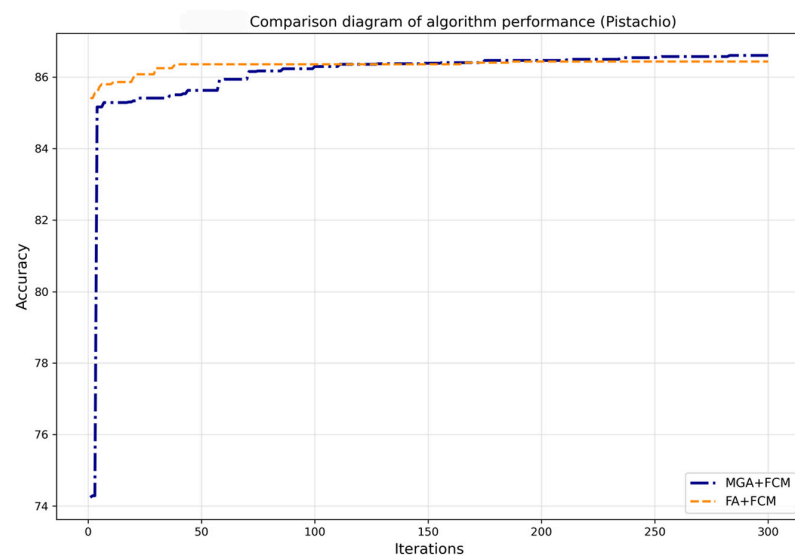
As shown in Table 5, the fuzzy controller based on the enhanced genetic algorithm is compared with the Firefly Algorithm [38] (FA) combined with FCM. On the Pistachio dataset, the proposed MGA + FCM method achieves an average accuracy of $86.61 \pm 0.06\%$, which is slightly higher than that of FA + FCM with $86.44 \pm 0.02\%$. On the WDBC dataset, MGA + FCM also obtains a higher average accuracy of $97.31 \pm 0.33\%$, compared with $97.07 \pm 0.08\%$ for FA + FCM. On the Wine dataset, FA + FCM achieves $98.31 \pm 0.46\%$, while MGA + FCM obtains $98.13 \pm 0.70\%$, showing comparable performance. Overall, the proposed MGA + FCM method performs better on two of the three datasets and remains competitive on the Wine dataset, indicating that the fuzzy controller based on the enhanced genetic algorithm is effective in classification tasks.

The final calculated feature weights of the proposed MGA + FCM method on the Pistachio dataset are shown in Table 6.

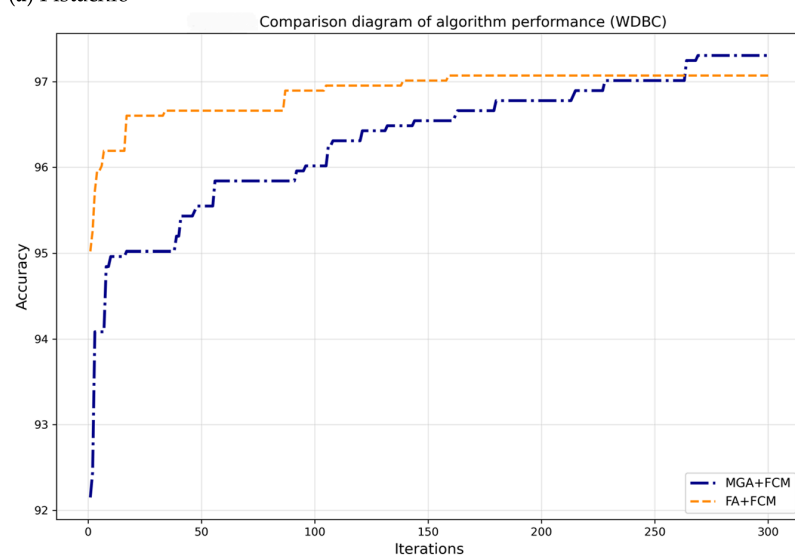
Table 6. Average feature weights of MGA + FCM on the Pistachio dataset.

k_1	k_2	k_3	k_4	k_5	k_6	k_7	k_8
0.0000	0.1721	0.0032	0.9319	0.5093	0.8034	0.6511	0.0000
k_9	k_{10}	k_{11}	k_{12}	k_{13}	k_{14}	k_{15}	k_{16}
0.2409	0.9628	0.5684	0.5381	0.2880	0.5324	0.2133	0.8434

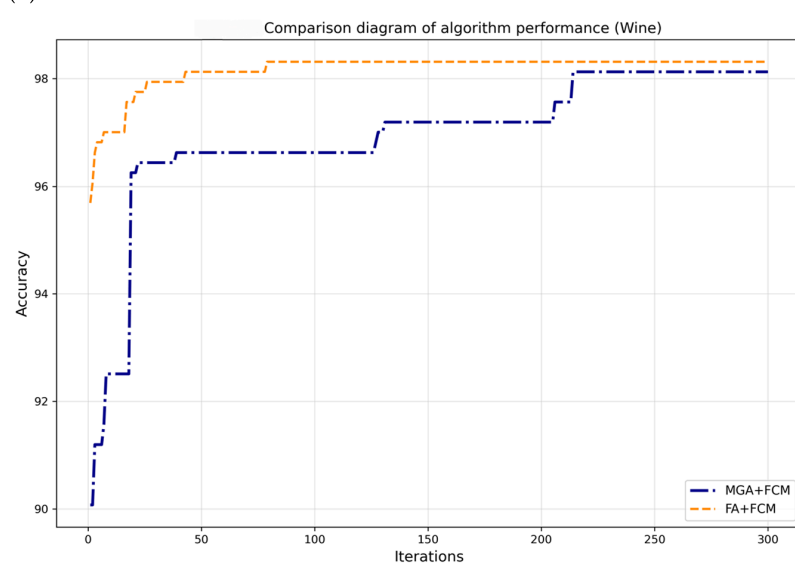
According to Table 6, the weights of k_1 and k_8 equal zero, and k_3 possesses an extremely low weight of merely 0.0032. By comparison, k_4 , k_6 , k_{10} , and k_{16} are assigned relatively large weights, with k_7 also attaining a considerably high weight value of 0.6511. These findings only partially align qualitatively with the fuzzy dynamic clustering analysis. Such discrepancy is reasonable: the dynamic clustering diagram primarily characterizes the similarity and clustering correlations between different features, while the optimized feature weights quantify each feature's individual contribution to the classification target. Figure 5 illustrates the convergence speed of all compared algorithms.



(a) Pistachio



(b) WDBC



(c) Wine

Figure 5. Convergence curves of MGA + FCM and FA + FCM on different datasets: (a) Pistachio; (b) WDBC; (c) Wine.

4.3. Ablation Study

To further evaluate the contribution of each component in the proposed enhanced genetic algorithm, an ablation study was conducted on the Pistachio dataset. The full MGA + FCM method was used as the baseline, and five ablated variants were constructed by removing one component at a time. Specifically, in the variant without the matrix structure, the matrix-structured population was flattened into a standard one-dimensional population. In the variant without the row search operator, the matrix structure was retained, but the row search operation was removed. In the variant without elite crossover, the crossover operation was removed. In the variant without adaptive mutation, a fixed mutation probability of 1% was used instead of the adaptive mutation strategy. In the variant without chaotic initialization, random initialization was used instead of Tent chaotic initialization.

All ablation variants were evaluated under the same experimental settings. The experiments were conducted with 150 iterations and repeated independently three times. The average accuracy and standard deviation are reported in Table 7.

Table 7. Ablation study results of the proposed MGA + FCM method.

Variant	Removed Component	Accuracy (%)	Performance Decrease	Importance Ranking
MGA + FCM	—	86.42 ± 0.10	—	Baseline
w/o Row Search	Row search operator	85.63 ± 0.39	−0.79%	1
w/o Elite Crossover	Elite crossover operator	85.88 ± 0.36	−0.54%	2
w/o Chaotic Init	Tent chaotic initialization	86.19 ± 0.15	−0.23%	3
w/o Matrix	Matrix-structured population	86.20 ± 0.32	−0.22%	4
w/o Adaptive Mutation	Adaptive mutation strategy	86.25 ± 0.09	−0.17%	5

As shown in Table 7, removing any individual component leads to a decrease in classification accuracy, indicating that all five components contribute to the performance of the proposed MGA + FCM method. Among them, removing the row search operator causes the largest decrease in accuracy, from 86.42% to 85.63%, with a performance drop of 0.79%. This suggests that the row search operator plays an important role in improving the population structure and preserving high-quality individuals during the evolutionary process. The removal of elite crossover results in the second-largest decrease, indicating that the crossover operation among elite individuals helps generate better candidate feature-weight solutions.

The effects of removing Tent chaotic initialization, the matrix structure, and the adaptive mutation strategy are relatively smaller, with performance decreases of 0.23%, 0.22%, and 0.17%, respectively. Although their individual effects are less pronounced than those of the row search operator and elite crossover, the accuracy still decreases after each component is removed. This shows that these components contribute to improving population diversity, maintaining structural organization, and adjusting the mutation process. Overall, the ablation results demonstrate that the proposed MGA + FCM method benefits from the combined effect of the five algorithmic components, rather than relying on a single modification.

5. Conclusions

This paper proposes a novel fuzzy controller based on an enhanced genetic algorithm (MGA + FCM) to address the critical issue of insufficient classification accuracy in traditional fuzzy C-means (FCM) clustering for multidimensional data. The proposed approach makes three key contributions that collectively improve clustering performance.

First, the enhanced genetic algorithm introduces several structural improvements. The Tent chaotic map is employed for population initialization, which ensures better diversity and uniform distribution of initial solutions in the search space, thereby avoiding premature convergence that often plagues traditional genetic algorithms. The matrix-structured population representation, combined with the best individual retention strategy and last elimination selection operator, effectively optimizes the population structure throughout the evolutionary process. Furthermore, the elitist crossover operator and adaptive tentative mutation operator work synergistically to maintain population diversity while accelerating convergence. The introduction of a nonlinear convergence factor, perturbed by beta-distributed random numbers, provides a dynamic balance between global exploration in early iterations and local exploitation in later stages, significantly mitigating the risk of falling into local optima.

Second, the fitness function design represents a methodological innovation by integrating intermediate parameters from the FCM algorithm—specifically classification accuracy, inter-cluster distance, and intra-cluster radius—into the genetic algorithm’s optimization framework. This integration enables the controller to screen optimal feature subsets while simultaneously performing fuzzy clustering, creating a feedback mechanism that enhances overall classification performance.

Third, experimental validation on the Pistachio, WDBC, and Wine datasets demonstrates that the proposed MGA + FCM method achieves competitive classification performance. Compared with FA + FCM, MGA + FCM obtains higher average accuracy on the Pistachio and WDBC datasets, and achieves comparable performance on the Wine dataset. These results indicate that the enhanced genetic algorithm can effectively search for feature-weight solutions under different data characteristics. The optimized feature weights reveal that k_1 and k_8 are effectively eliminated (zero weights), while k_3 receives a negligible weight of 0.0032. On the other hand, k_4 , k_6 , k_{10} , and k_{16} emerge as the most influential features, with k_7 also demonstrating substantial importance at 0.6511. Such weight distributions exhibit merely partial qualitative agreement with the fuzzy dynamic clustering analysis, which is expected since the clustering graph captures feature similarity and redundancy patterns, whereas the learned weights reflect discriminative power for the classification task.

In summary, the fuzzy controller based on the enhanced genetic algorithm not only improves classification accuracy through optimized feature weighting but also provides a robust framework for handling multidimensional data clustering. The combination of chaotic population initialization, nonlinear adaptive mechanisms, and FCM-integrated fitness evaluation offers a promising direction for intelligent data classification in information-driven applications. Future work may extend this approach to larger-scale datasets and explore its applicability to other clustering domains beyond binary classification problems.

Author Contributions: Conceptualization, Z.L. and Y.Z.; methodology, Z.L. and X.W.; software, Z.L.; validation, Z.L. and X.W.; formal analysis, Z.L., X.W. and Y.L.; data curation, X.W. and Y.L.; writing—original draft preparation, Z.L.; writing—review and editing, X.W., Y.L. and Y.Z.; visualization, Z.L. and Y.L.; supervision, Y.Z.; project administration, Y.Z.; funding acquisition, Y.Z. All authors have read and agreed to the published version of the manuscript.

Funding: This work was supported by National Natural Science Foundation of China (No. 12250410247), Guangzhou University—Guangdong Oriental Thinking Technology Co., LTD. Science, Industry and Education Integration Practice Teaching Base Fund Project (202282) of Guangdong Provincial Department of Education, 2025 Guangzhou Higher Education Teaching Quality and Teaching Reform Project (2025YBJG013).

Data Availability Statement: The Pistachio dataset is openly available at Kaggle: <https://www.kaggle.com/datasets/muratkokludataset/pistachio-dataset> (accessed on 16 April 2026). The WDBC and Wine datasets are publicly available from the UCI Machine Learning Repository: <https://doi.org/10.24432/C5DW2B> (accessed on 16 June 2026) and <https://doi.org/10.24432/C5PC7J> (accessed on 16 June 2026).

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

FCM	Fuzzy C-means clustering
GA	Genetic Algorithm
MGA	Enhanced Genetic Algorithm
FA	Firefly Algorithm
MGA + FCM	Enhanced Genetic Algorithm combined with Fuzzy C-means clustering
GA + FCM	Genetic Algorithm combined with Fuzzy C-means clustering
FA + FCM	Firefly Algorithm combined with Fuzzy C-means clustering

References

- Dunn, J.C. A Fuzzy Relative of the ISODATA Process and Its Use in Detecting Compact Well-Separated Clusters. *J. Cybern.* **1973**, *3*, 32–57. [CrossRef]
- Chen, M.; Mao, S.; Liu, Y. Big data: A survey. *Mob. Netw. Appl.* **2014**, *19*, 171–209. [CrossRef]
- Jain, A.K. Data clustering: 50 years beyond k-means. *Pattern Recognit. Lett.* **2010**, *31*, 651–666. [CrossRef]
- Yao, X.Q.; Li, Y.Z.; Xu, Y.; Chen, J.H.; Zhou, P. Comprehensive evaluation of air quality in Chengdu City based on entropy weight and cluster analysis methods. *Environ. Prot. Sci.* **2017**, *43*, 100–104.
- Xu, R.; Wunsch, D. Survey of clustering algorithms. *IEEE Trans. Neural Netw.* **2005**, *16*, 645–678. [CrossRef] [PubMed]
- MacQueen, J. Some methods for classification and analysis of multivariate observations. In *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*; University of California Press: Berkeley, CA, USA, 1967; Volume 1, pp. 281–297.
- Bezdek, J.C. *Pattern Recognition with Fuzzy Objective Function Algorithms*; Plenum Press: New York, NY, USA, 1981.
- Selim, S.Z.; Ismail, M.A. K-means-type algorithms: A generalized convergence theorem and characterization of local optimality. *IEEE Trans. Pattern Anal. Mach. Intell.* **1984**, *6*, 81–87. [CrossRef] [PubMed]
- Wu, K.L.; Yang, M.S. Alternative c-means clustering algorithms. *Pattern Recognit.* **2002**, *35*, 2267–2278. [CrossRef]
- Höppner, F.; Klawonn, F.; Kruse, R.; Runkler, T. *Fuzzy Cluster Analysis: Methods for Classification, Data Analysis and Image Recognition*; John Wiley & Sons: Chichester, UK, 1999.
- Goldberg, D.E. *Genetic Algorithms in Search, Optimization, and Machine Learning*; Addison-Wesley: Boston, MA, USA, 1989.
- Srinivas, M.; Patnaik, L.M. Adaptive probabilities of crossover and mutation in genetic algorithms. *IEEE Trans. Syst. Man Cybern.* **1994**, *24*, 656–667. [CrossRef]
- Bäck, T. *Evolutionary Algorithms in Theory and Practice: Evolution Strategies, Evolutionary Programming, Genetic Algorithms*; Oxford University Press: New York, NY, USA, 1996.
- May, R.M. Simple mathematical models with very complicated dynamics. *Nature* **1976**, *261*, 459–467. [CrossRef] [PubMed]
- Tavazoei, M.S.; Haeri, M. Comparison of different one-dimensional maps as chaotic search pattern in chaos optimization algorithms. *Appl. Math. Comput.* **2007**, *187*, 1076–1085. [CrossRef]
- Yang, C.R.; Qian, Q.; Wang, F.; Sun, M.H. Application of improved adaptive genetic algorithm in function optimization. *Appl. Res. Comput.* **2018**, *35*, 1042–1045.
- Agrawal, P.; Abutarboush, H.F.; Ganesh, T.; Mohamed, A.W. Metaheuristic algorithms on feature selection: A survey of one decade of research (2009–2019). *IEEE Access* **2021**, *9*, 35666–35691. [CrossRef]
- Maulik, U.; Bandyopadhyay, S. Genetic algorithm-based clustering technique. *Pattern Recognit.* **2000**, *33*, 1455–1465. [CrossRef]
- Yang, D.; Li, G.; Cheng, G. On the efficiency of chaos optimization algorithms based on ergodic theory. *Chaos Solitons Fractals* **2007**, *34*, 1366–1375. [CrossRef]
- Sathya, P.D.; Kayalvizhi, R. Modified bacterial foraging algorithm based multilevel thresholding for image segmentation. *Eng. Appl. Artif. Intell.* **2011**, *24*, 595–615. [CrossRef]
- Karaboga, D.; Basturk, B. A powerful and efficient algorithm for numerical function optimization: Artificial bee colony (ABC) algorithm. *J. Glob. Optim.* **2007**, *39*, 459–471. [CrossRef]

22. Nadimi-Shahraki, M.H.; Taghian, S.; Mirjalili, S. An improved grey wolf optimizer for solving engineering problems. *Expert Syst. Appl.* **2021**, *166*, 113917. [[CrossRef](#)]
23. Faris, H.; Mafarja, M.M.; Heidari, A.A.; Aljarah, I.; Ala'M, A.Z.; Mirjalili, S.; Fujita, H. An efficient binary salp swarm algorithm with crossover scheme for feature selection problems. *Knowl.-Based Syst.* **2018**, *154*, 43–67. [[CrossRef](#)]
24. Qi, Y.; Jiang, A.; Gao, Y. A Gaussian convolutional optimization algorithm with tent chaotic mapping. *Sci. Rep.* **2024**, *14*, 31027. [[CrossRef](#)] [[PubMed](#)]
25. Liu, M.; Li, C.; Zhang, Y.; Guo, J.; Chen, J.; Liu, Z. An adaptive lion swarm optimization algorithm incorporating tent chaotic search and information entropy. *Sci. Rep.* **2023**, *13*, 4661.
26. Rauf, H.T.; Gao, J.; Almadhor, A.; Haider, A.; Zhang, Y.-D.; Al-Turjman, F. Multi-Population-Based Chaotic Differential Evolution for Multi-Modal and Multi-Objective Optimization Problems. *Appl. Soft Comput.* **2023**, *134*, 109909. [[CrossRef](#)]
27. Wang, Y.; Zhang, X.; Yu, D.J.; Bai, Y.J.; Du, J.P.; Tian, Z.T. Tent chaotic map and population classification evolution strategy-based dragonfly algorithm for global optimization. *Math. Probl. Eng.* **2022**, *2022*, 2508414. [[CrossRef](#)]
28. Askari, S. Fuzzy C-Means clustering algorithm for data with unequal cluster sizes and contaminated with noise and outliers: Review and development. *Expert Syst. Appl.* **2021**, *165*, 113856. [[CrossRef](#)]
29. Liu, M.; Chen, H.; Wang, J. A hybrid particle swarm clustering algorithm with novel fitness function based on minimum spanning tree and local centroids. *Sci. Rep.* **2025**, *15*, 378. [[CrossRef](#)]
30. Zheng, K.; Han, L.; Guo, S.; Wang, Z.; Zhang, X.; Dong, X. Fuzzy synthetic condition assessment of wind turbine based on combination weighting and cloud model. *J. Intell. Fuzzy Syst.* **2017**, *32*, 4563–4572. [[CrossRef](#)]
31. De Castro, L.N.; Von Zuben, F.J. The clonal selection algorithm with engineering applications. In Proceedings of the GECCO 2000 Workshop on Artificial Immune Systems and Their Applications, Las Vegas, NV, USA, 8–12 July 2000; pp. 36–37.
32. Gu, H.; Fu, X.; Zhu, Y.; Chen, Y.; Huang, L. Analysis of social and environmental impact of earth-rock dam breaks based on a fuzzy comprehensive evaluation method. *Sustainability* **2020**, *12*, 6239. [[CrossRef](#)]
33. He, Q.; Wu, Y.L.; Xu, T.W. Application of improved genetic simulated annealing algorithm in TSP optimization. *Control Decis.* **2018**, *33*, 219–225.
34. Zhao, Y.; Xu, X.; Zhao, Y.; Chu, X.N. Research on adaptive memory genetic algorithm. *Comput. Technol. Dev.* **2014**, *24*, 63–66.
35. Koklu, M. Pistachio Dataset. Kaggle. Available online: <https://www.kaggle.com/datasets/muratkokludataset/pistachio-dataset> (accessed on 2 May 2026).
36. Wolberg, W.; Mangasarian, O.; Street, N.; Street, W. *Breast Cancer Wisconsin (Diagnostic) [Dataset]*; UCI Machine Learning Repository: Irvine, CA, USA, 1993. [[CrossRef](#)]
37. Aeberhard, S.; Forina, M. *Wine [Dataset]*; UCI Machine Learning Repository: Irvine, CA, USA, 1992. [[CrossRef](#)]
38. Yang, X.-S. Firefly Algorithms for Multimodal Optimization. In *Stochastic Algorithms: Foundations and Applications*; Watanabe, O., Zeugmann, T., Eds.; Lecture Notes in Computer Science; Springer: Berlin/Heidelberg, Germany, 2009; Volume 5792, pp. 169–178.

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.